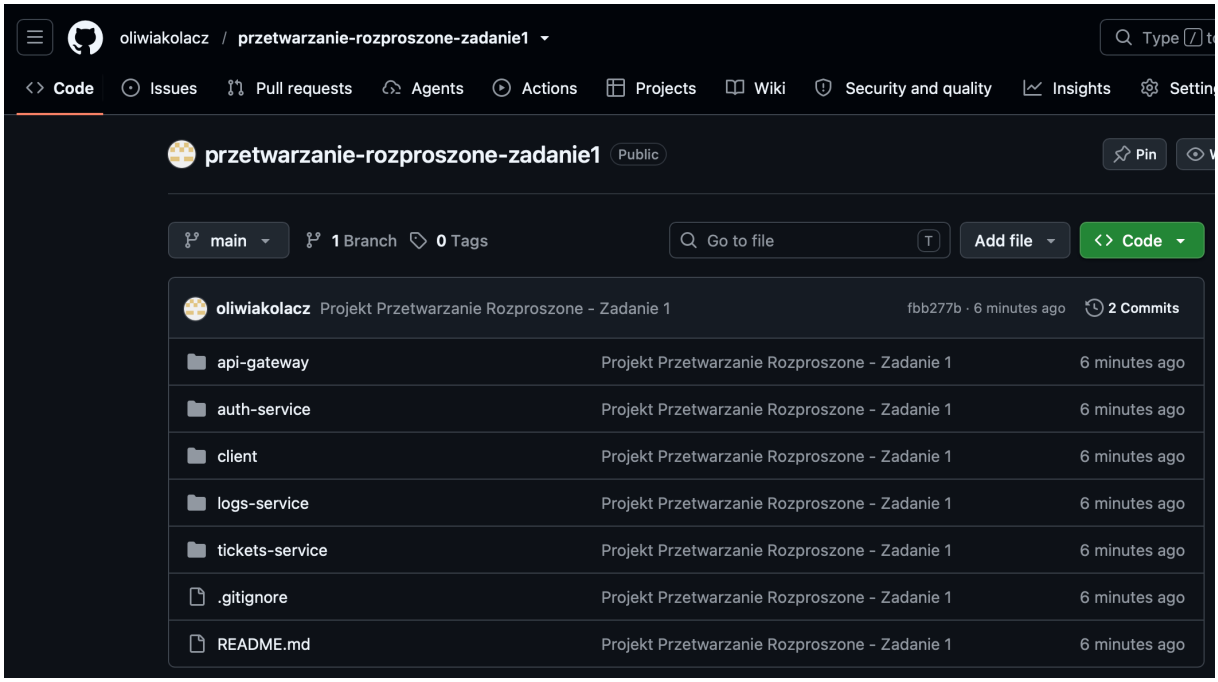
Oliwia Kołacz (indeks 83099)

Przetwarzanie rozproszone – Zadanie 1 - System zgłoszeń

GitHub

https://github.com/oliwiakolacz/przetwarzanie-rozproszone-zadanie1

Struktura projektu



Opis

System obsługi zgłoszeń (CRUD) oparty na 4 serwisach: api-gateway, auth-service, tickets-service, logs-service i kliencie (aplikacja webowa). Komunikacja odbywa się przez api-gateway (założono że tylko ten serwis jest publiczny). Token JWT jest uzyskiwany podczas logowania użytkownika, trzymany jest w sessionStorage i dołączany do każdego żądania w nagłówku: Authorization: Bearer <token>.

Technologie: Node.js 22, Express, SQLite (wbudowany moduł node:sqlite), JWT (jsonwebtoken), bcrypt (bcryptjs), http-proxy-middleware.

Komponenty

Komponent	Port	Rola	Baza danych
api-gateway	8080	jedyny punkt wejścia, routing, sprawdzanie tokena, klient www	–
auth-service	3001	rejestracja, logowanie, wydawanie i weryfikacja JWT	`users.db`
tickets-service	3002	CRUD zgłoszeń	`tickets.db`
logs-service	3003	zapis i odczyt logów zdarzeń	`logs.db`
client	–	strona www (HTML, JS) udostępniana przez gateway	–

Endointy dostępne przez api-gateway

Metoda	Ścieżka	Token	Opis
POST	/api/auth/register	nie	rejestracja
POST	/api/auth/login	nie	logowanie, token JWT
GET	/api/tickets	tak	lista zgłoszeń
GET	/api/tickets/:id	tak	jedno zgłoszenie
POST	/api/tickets	tak	dodanie zgłoszenia
PUT	/api/tickets/:id	tak	edycja zgłoszenia
DELETE	/api/tickets/:id	tak	usunięcie zgłoszenia
GET	/api/logs[?service=]	tak	podgląd logów

Weryfikacja tokena przez api-gateway

api-gateway/index.js | auth-service/index.js | logs-service/index.js | tickets-service/index.js | index.html

```
// Sprawdzenie tokena, pytamy serwis autoryzacji (auth-service)
async function requireAuth(req, res, next) : Promise<...> { Show usages
  try {
    const response : Response = await fetch( input: `${AUTH_URL}/verify`, init: {
      headers: { authorization: req.headers.authorization || '' },
    });
    if (!response.ok) return res.status(401).json({ error: 'Nieautoryzowany - zaloguj się' });

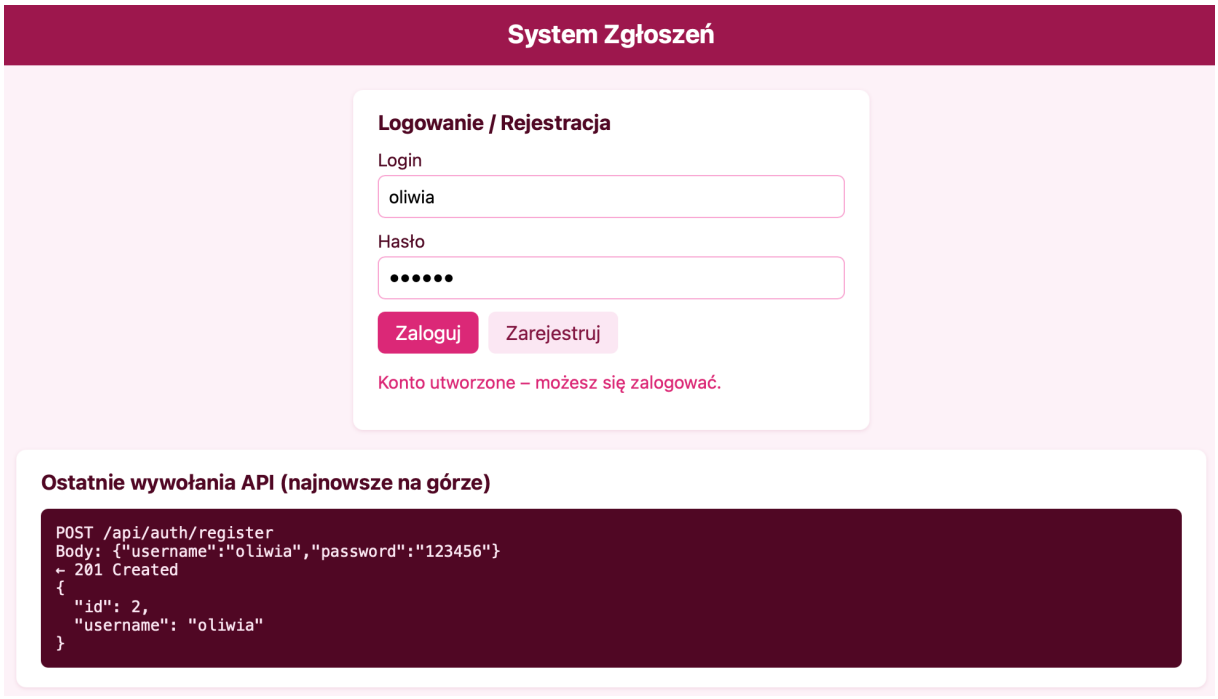
    const user = await response.json();
    // Przekazujemy dalej informację, kto wykonuje żądanie
    req.headers['x-user-id'] = String(user.id);
    req.headers['x-username'] = user.username;
    next();
  } catch (err) {
    res.status(503).json({ error: 'Serwis autoryzacji niedostępny' });
  }
}
```

Routing do serwisów przez api-gateway

```
// Routing
app.use('/api/auth',    proxy(AUTH_URL,    rewrite: { '^/': '/' }));                  // /api/auth/login -> /login
app.use('/api/tickets', requireAuth, proxy(TICKETS_URL, rewrite: { '^/': '/tickets/' })); // /api/tickets/5 -> /tickets/5
app.use('/api/logs',    requireAuth, proxy(LOGS_URL,    rewrite: { '^/': '/logs/' }));    // /api/logs -> /logs/

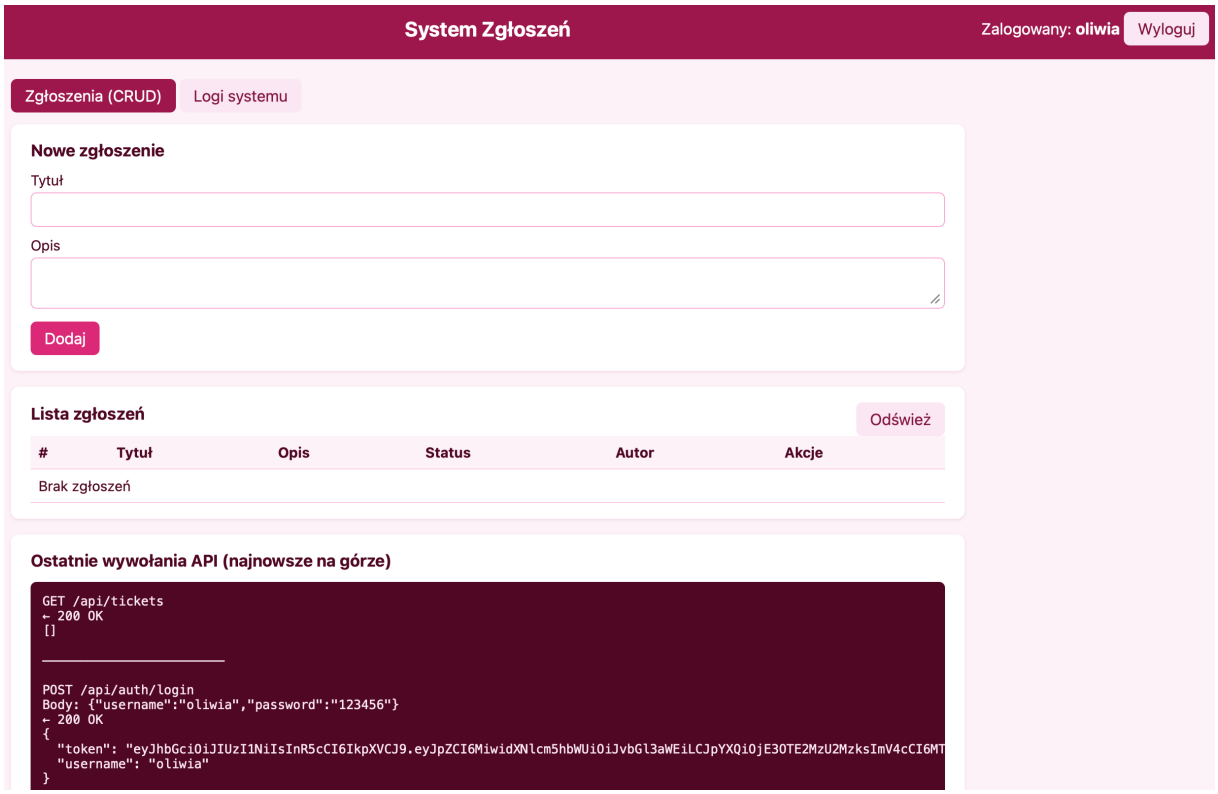
```

Rejestracja (POST /api/auth/register)

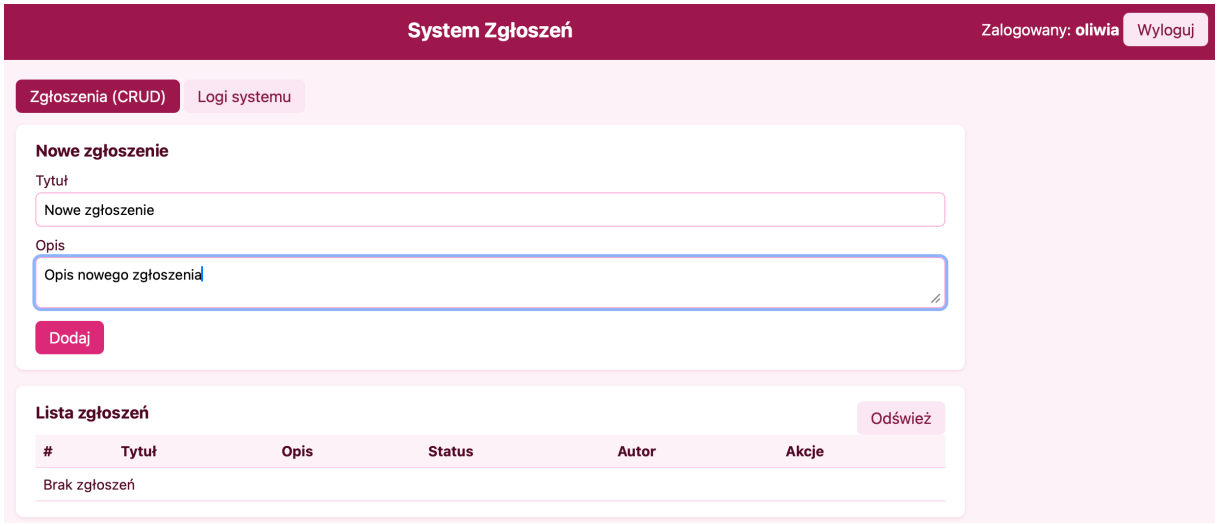


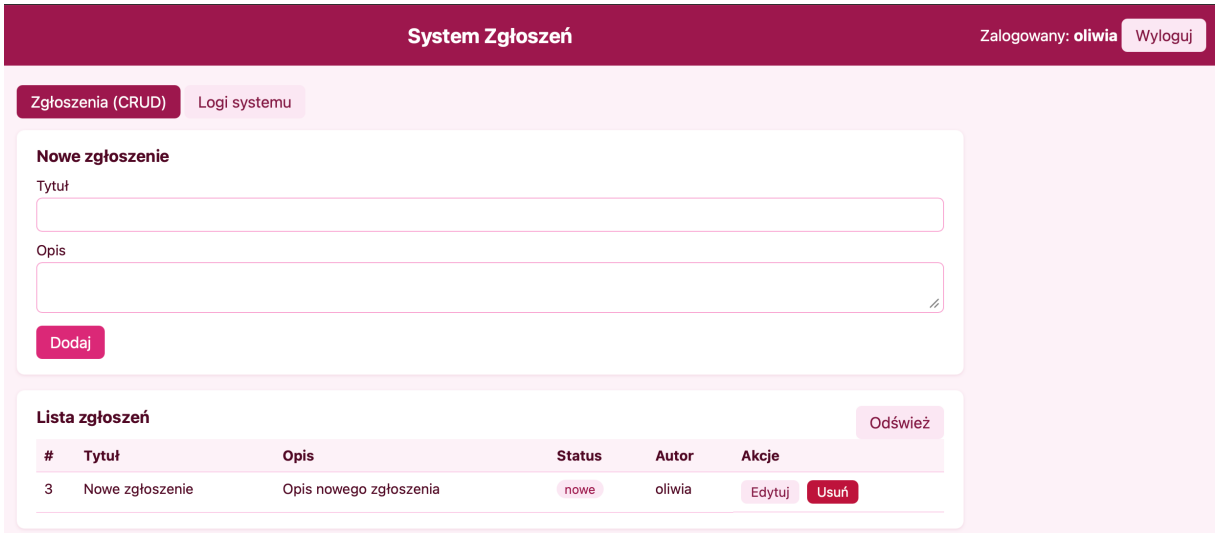
Logowanie i generowanie tokena (POST /api/auth/login)





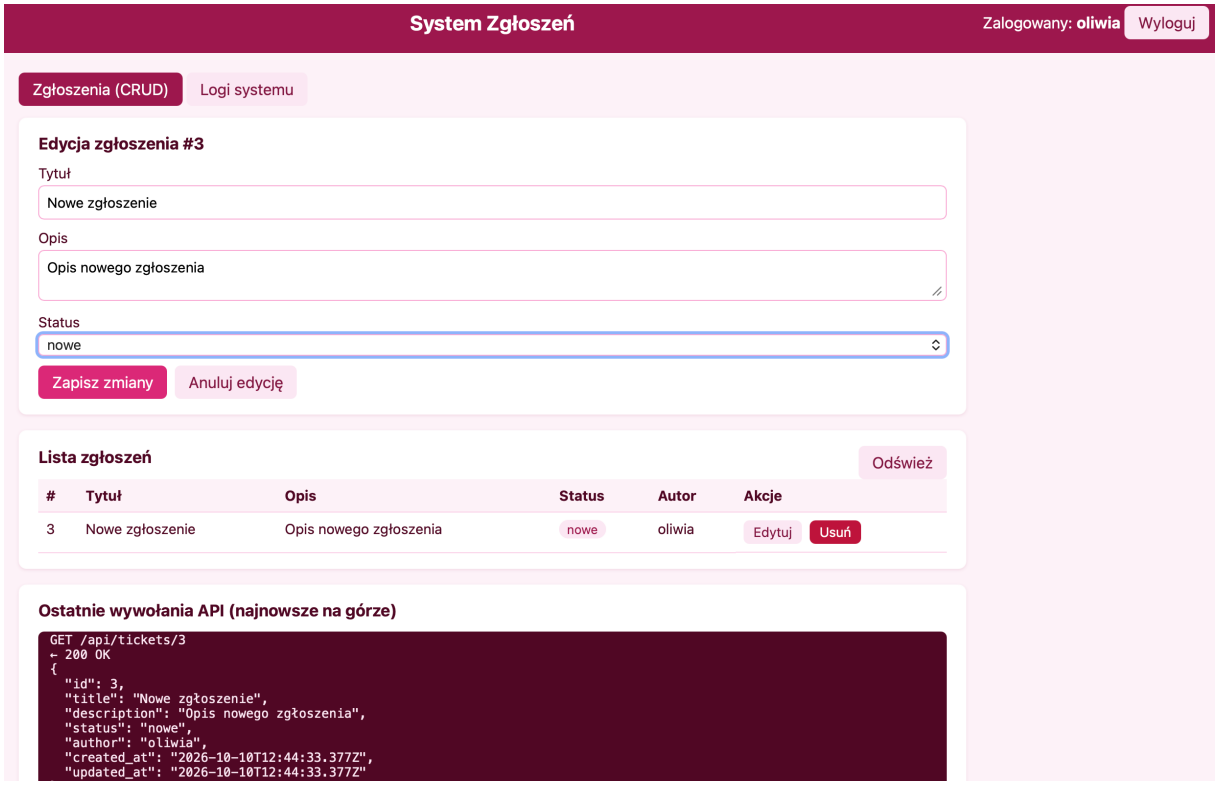
Tworzenie zgłoszenia (POST /api/tickets)

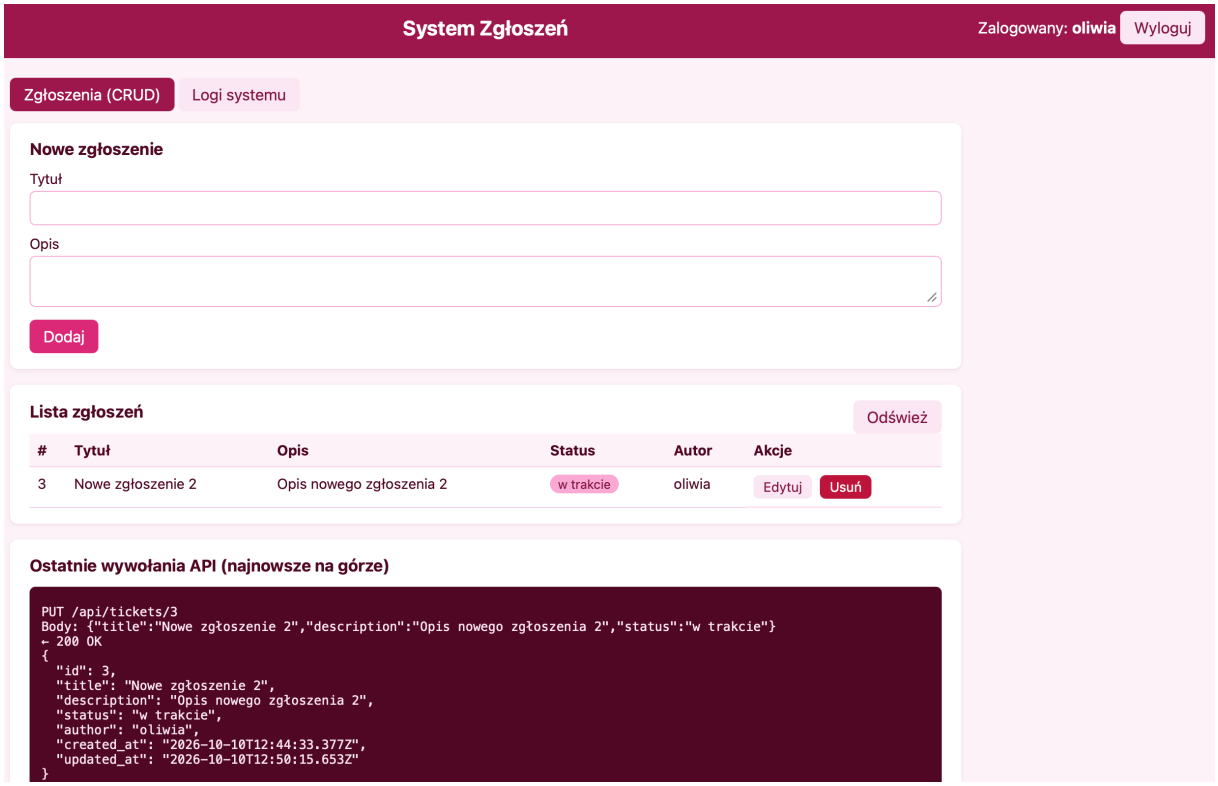




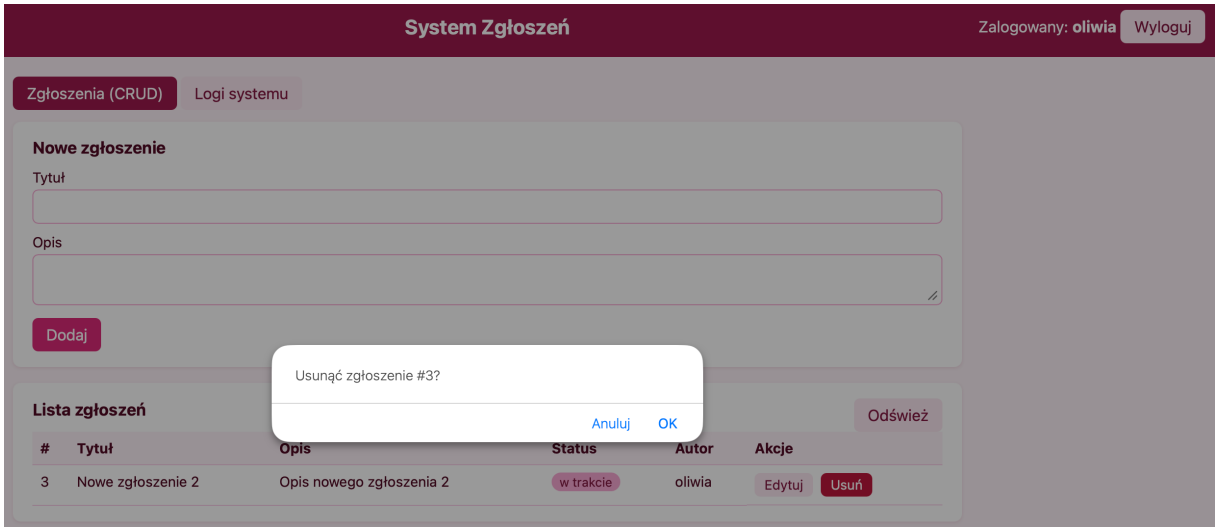


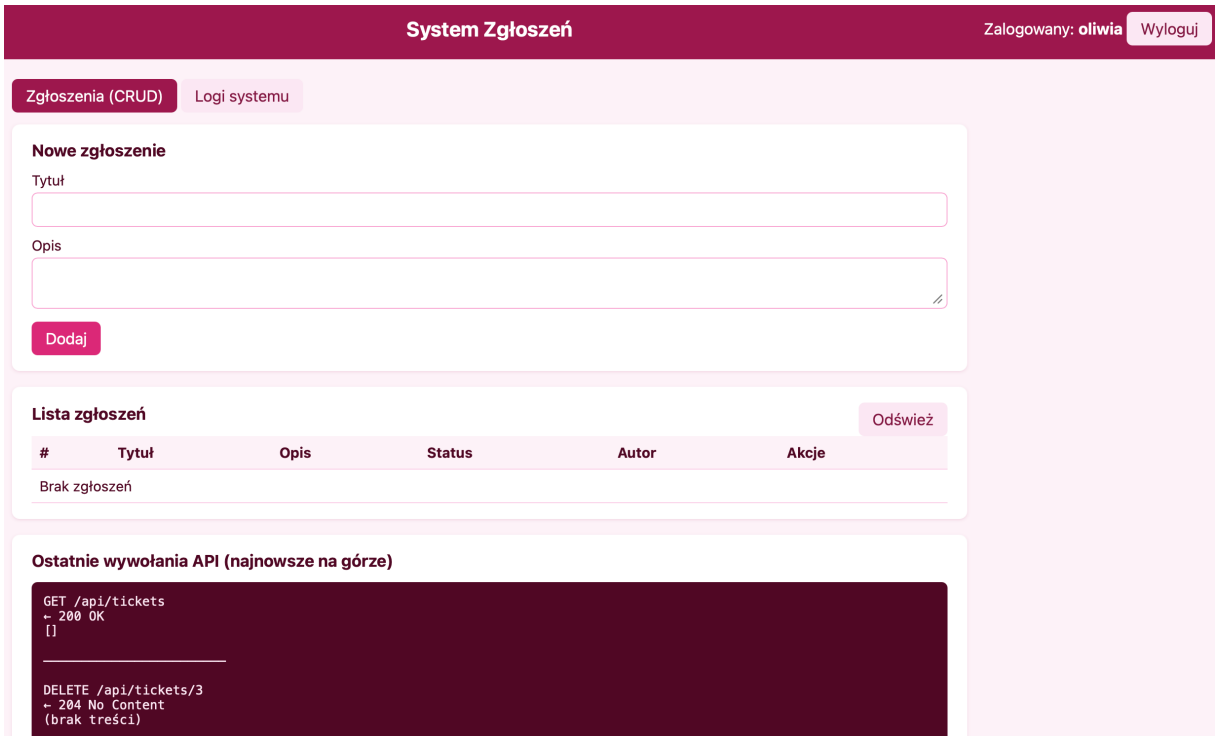
Edycja zgłoszenia (PUT /api/tickets/:id)



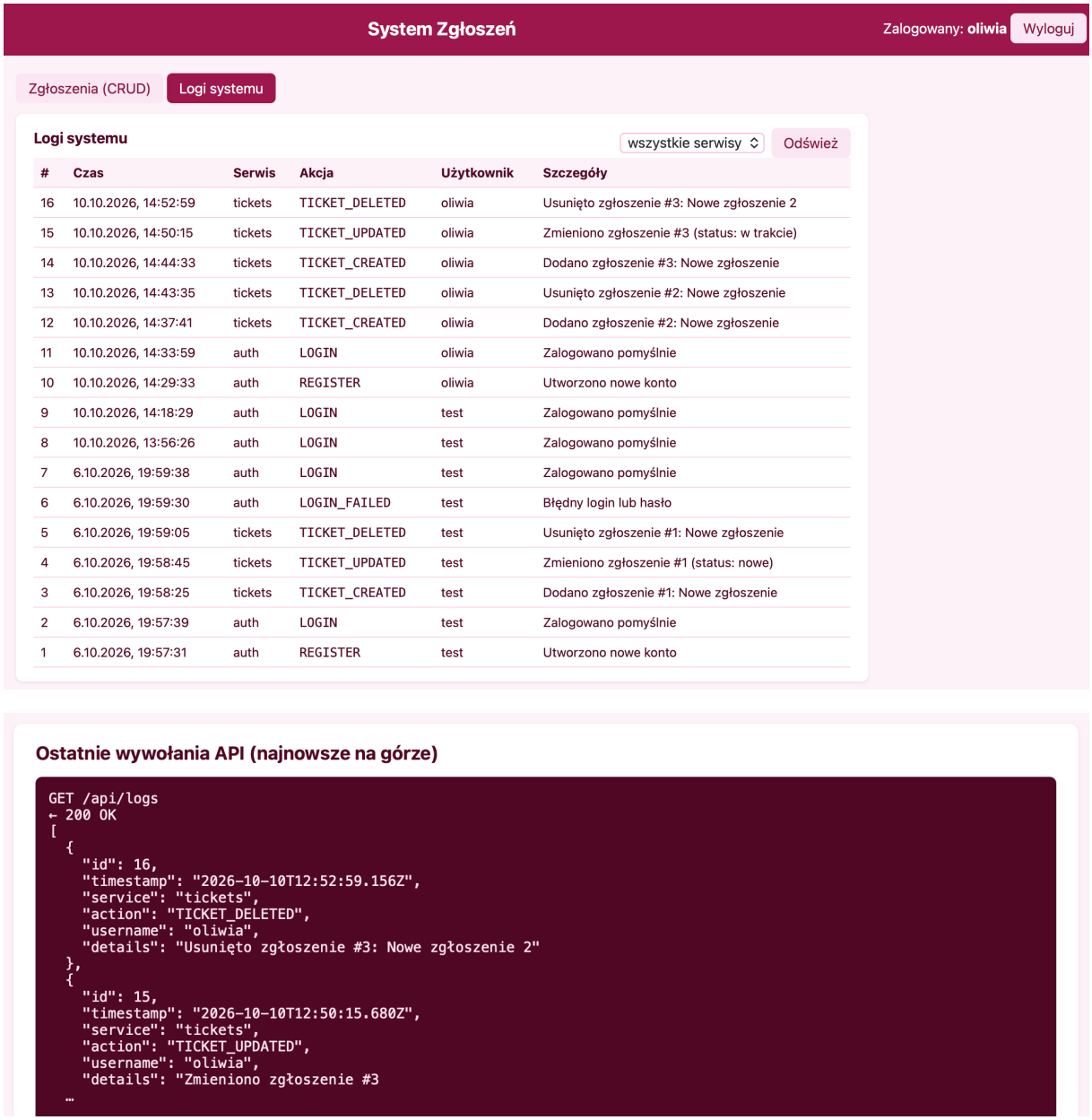


Usuwanie zgłoszenia (DELETE /api/tickets/:id)





Logi (GET /api/logs)



Ostatnie wywołania API (najnowsze na górze)

```
GET /api/logs
← 200 OK
[
  {
    "id": 16,
    "timestamp": "2026-10-10T12:52:59.156Z",
    "service": "tickets",
    "action": "TICKET_DELETED",
    "username": "oliwia",
    "details": "Usunięto zgłoszenie #3: Nowe zgłoszenie 2"
  },
  {
    "id": 15,
    "timestamp": "2026-10-10T12:50:15.680Z",
    "service": "tickets",
    "action": "TICKET_UPDATED",
    "username": "oliwia",
    "details": "Zmieniono zgłoszenie #3
  …
```